

```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    precision_recall_fscore_support
)

# -----
# STEP 1 : Load Dataset
# -----

data = pd.read_csv("pos_tags (1).csv")

# Remove missing values
data = data.dropna()

print(data.head())
print(data.columns)

# -----
# STEP 2 : Convert Dataset into Sentences
# -----

sentences = []

for _, group in data.groupby("sentence_id"):

    sentence = []

    for _, row in group.iterrows():
        word = str(row["word"]).lower()
        tag = row["tag"]

        sentence.append((word, tag))

    sentences.append(sentence)

print("Total Sentences :", len(sentences))

# -----
# STEP 3 : Train Test Split

```

```

# -----

train_sentences, test_sentences = train_test_split(
    sentences,
    test_size=0.2,
    random_state=42
)

print("Training Sentences :", len(train_sentences))
print("Testing Sentences :", len(test_sentences))

# -----
# STEP 4 : Vocabulary and Tags
# -----

tags = sorted(list(set(tag for sent in train_sentences for _, tag in sent)))
words = sorted(list(set(word for sent in train_sentences for word, _ in sent)))

tag_to_idx = {tag: i for i, tag in enumerate(tags)}
idx_to_tag = {i: tag for tag, i in tag_to_idx.items()}

word_to_idx = {word: i for i, word in enumerate(words)}

num_tags = len(tags)
num_words = len(words)

print("Number of Tags :", num_tags)
print("Number of Words :", num_words)

# -----
# STEP 5 : Initial, Transition and Emission Counts
# -----

initial = np.ones(num_tags)

transition = np.ones((num_tags, num_tags))

emission = np.ones((num_tags, num_words))

for sentence in train_sentences:

    first_tag = sentence[0][1]
    initial[tag_to_idx[first_tag]] += 1

```

```

for i, (word, tag) in enumerate(sentence):

    tag_index = tag_to_idx[tag]

    emission[tag_index, word_to_idx[word]] += 1

    if i > 0:
        previous_tag = sentence[i - 1][1]
        previous_index = tag_to_idx[previous_tag]

        transition[previous_index, tag_index] += 1

# -----
# STEP 6 : Probability Matrices
# -----

initial = initial / initial.sum()

transition = transition / transition.sum(axis=1, keepdims=True)

emission = emission / emission.sum(axis=1, keepdims=True)

print("\nInitial Matrix Shape :", initial.shape)
print("Transition Matrix Shape :", transition.shape)
print("Emission Matrix Shape :", emission.shape)

# -----
# STEP 7 : Log Space
# -----
g
log_initial = np.log(initial)
log_transition = np.log(transition)
log_emission = np.log(emission)

# -----
# STEP 8 : Vectorized Viterbi Algorithm
# -----

def viterbi(sentence):

    T = len(sentence)

    dp = np.full((num_tags, T), -np.inf)

```

```

back = np.zeros((num_tags, T), dtype=int)

first_word = sentence[0].lower()

if first_word in word_to_idx:
    emission_prob = log_emission[:, word_to_idx[first_word]]
else:
    emission_prob = np.log(np.ones(num_tags) / num_words)

dp[:, 0] = log_initial + emission_prob

for t in range(1, T):

    word = sentence[t].lower()

    if word in word_to_idx:
        emission_prob = log_emission[:, word_to_idx[word]]
    else:
        emission_prob = np.log(np.ones(num_tags) / num_words)

    scores = dp[:, t - 1][:, None] + log_transition

    back[:, t] = np.argmax(scores, axis=0)

    dp[:, t] = np.max(scores, axis=0) + emission_prob

best_path = np.zeros(T, dtype=int)

best_path[-1] = np.argmax(dp[:, -1])

for t in range(T - 2, -1, -1):
    best_path[t] = back[best_path[t + 1], t + 1]

return [idx_to_tag[i] for i in best_path]

# -----
# STEP 9 : Prediction
# -----

true_tags = []
predicted_tags = []

for sentence in test_sentences:

```

```

words_only = [word for word, tag in sentence]

actual = [tag for word, tag in sentence]

predicted = viterbi(words_only)

true_tags.extend(actual)
predicted_tags.extend(predicted)

# -----
# STEP 10 : Evaluation
# -----

accuracy = accuracy_score(true_tags, predicted_tags)

precision, recall, f1, _ = precision_recall_fscore_support(
    true_tags,
    predicted_tags,
    average="weighted",
    zero_division=0
)

print("\n=====")
print("Evaluation")
print("=====")

print(f"Accuracy : {accuracy*100:.2f}%")
print(f"Precision : {precision*100:.2f}%")
print(f"Recall : {recall*100:.2f}%")
print(f"F1 Score : {f1*100:.2f}%")

print("\nClassification Report\n")
print(classification_report(true_tags, predicted_tags, zero_division=0))

# -----
# STEP 11 : Test on Unseen Sentences
# -----

test_examples = [
    "I love machine learning".split(),
    "She is reading a book".split(),
    "The dog barked loudly".split(),
    "We are going to college".split(),
    "Artificial intelligence changes the world".split()

```

```
print("\n=====")
print("Predictions on Unseen Sentences")
print("=====")
```

```
for sentence in test_examples:
```

```
prediction = viterbi(sentence)
```

```
print("\nSentence :", " ".join(sentence))
```

```
for word, tag in zip(sentence, prediction):
```

```
print(f"{word:<20} {tag}")
```

```

sentence_id  word  tag
0           0    aa  NN
1           1    aa  NN
2           2    aa  NN
3           3  added  VBN
4           4  adding  VBG
index['sentence_id', 'word', 'tag', dtype='object']
total sentences : 40000
training sentences : 29000
testing sentences : 10000
number of tags : 20
number of words : 290000

```

```
Initial Matrix Shape : (24,)  
Transition Matrix Shape : (24, 24)  
Emission Matrix Shape : (24, 260000)
```

```

Evaluation
Accuracy : 82.88%
Precision : 88.88%
Recall : 82.88%
F1 Score : 87.80%

```

Classification Report

[illegible]

Predictions on Unseen Sentences

```

Sentence : I love machine learning
I          NN
love       CC
machine    NN
learning   VB

```

sentence :	she is reading a book
she	NN
is	VBZ
reading	NN
a	CC
book	NN

```

sentence : The dog barked loudly
the      NN
dog      NN
barked   VBN
loudly   RB

```

```
sentence : we are going to college
we          NN
are         VBP
going      CC
to         CC
college    NN
```

sentence :	Artificial intelligence changes the world
Artificial	NN
intelligence	NN
changes	NN
the	CC
world	NN